

TAREA 1 · TRABAJO INDIVIDUAL · GODOT 4.x · 100 PUNTOS

Escena base 3D controlable y modelo integrado

CharacterBody3D · SpringArm3D · FSM Idle/Walk/Jump · circuito de parkour

Datos de la entrega

Alumno	Alan Zenteno Paz	Matrícula	2203065778
UEA	Seminario de Sistemas de Información II	Fecha	[07/10/2026]
Repositorio / proyecto completo	https://github.com/AlanZentenoPaz/Tarea-01.git	Video (1–2 min)	https://drive.google.com/file/d/1NU_o8MFLmxKdK0e4eb_czuQ9ZIR6If34/view?usp=sharing

Resumen

Este reporte documenta una escena 3D en Godot 4.x en la que un personaje (**hunk.glb**) se controla con un **CharacterBody3D** sobre un escenario de calle de estilo industrial (**low_poly_street_gameready_6.glb**) ampliado con un circuito de parkour: escaleras, plataformas, pilares, cajas, muros bajos y barriles. El trabajo se centra en separar con claridad cuatro responsabilidades: **movimiento**, **física** (StaticBody3D y paso fijo de 60 Hz), **cámara** (SpringArm3D) y **apariencia** (el modelo vive en una escena envolvente que el controlador no necesita conocer).

El personaje se mueve en relación con la cámara, salta solo en el suelo y cambia entre Idle, Walk y Jump con transiciones explícitas visibles en pantalla. Brazos y piernas se animan por código y la cabeza sigue a la cámara; los clips definitivos quedan para la Tarea 2.

Cobertura de la guía de evaluación

Criterio	Pts.	Dónde se resuelve
Entorno, iluminación y colisiones	20	Level.tscn: cielo procedural, DirectionalLight3D con sombras, StaticBody3D con cajas y cilindros
Controlador y tratamiento del tiempo	25	player.gd: física a paso fijo, delta aplicado una sola vez, interpolación visual
Cámara	10	camera_rig.gd + SpringArm3D con forma esférica y exclusión del jugador
FSM y transiciones	15	player.gd (match de estados) + HUD con registro de transiciones
Inspección e integración del modelo	20	HunkModel.tscn (envoltorio), ficha de modelos, escala y orientación corregidas
Pruebas, documentación y trazabilidad	10	Este reporte, README, docs/FICHA_MODELOS.md y Prompts/BITACORA.md

Procedencia: recurso de práctica de Sketchfab (Hunk, CC-BY-4.0, Vasian-Digital3D) y escenario "low poly street gameready (6)"; no se generó un modelo con Tripo o Meshy. Detalles en docs/FICHA_MODELOS.md y Prompts/BITACORA.md.

1. Estructura de escenas

El proyecto se divide en escenas pequeñas y editables. Todas se pueden abrir y modificar desde la vista 3D del editor, y el modelo importado nunca se mezcla con la lógica del controlador.

Main.tscn (Node3D) - Level [instancia de Level.tscn] - Player [instancia de Player.tscn] - HUD [instancia de HUD.tscn]		Player.tscn - CharacterBody3D - CollisionShape3D player.gd cápsula r=0.35 h=1.8 (top_level, solo visual) - ModelPivot - HunkModel hunk_visual.gd instancia de hunk.glb - Model camera_rig.gd (top_level) - CameraRig - Pitch - SpringArm3D - Camera3D
Level.tscn - WorldEnvironment cielo procedural - Sun DirectionalLight3D + sombras - Street instancia del .glb (escala x2) - SpawnPoint Marker3D (grupo "spawn") - Colliders suelo, límites y obstáculos - StaticBody3D + Shape - Parkour escaleras, plataformas, pilares, cajas, muros, barriles		HUD.tscn - CanvasLayer hud.gd - StateLabel LogLabel - TextLabel HelpLabel

Responsabilidades por archivo

Archivo	Responsabilidad
player.gd	Entrada, gravedad, salto, movimiento relativo a la cámara, FSM, prueba de desplazamiento y posición interpolada para dibujar.
camera_rig.gd	Yaw y pitch de la cámara (mouse y flechas), seguimiento suave del jugador y exclusión del cuerpo en el SpringArm3D.
hunk_visual.gd	Pose procedural del esqueleto: brazos, piernas, torso y cabeza. No toca la física.
hud.gd	Estado activo, registro de transiciones, FPS, límite de render y resultado de la prueba.
input_setup.gd	Autoload que registra las acciones de teclado por código.

Entorno, iluminación y colisiones

El cielo es un **ProceduralSkyMaterial** y la luz es una **DirectionalLight3D** con sombras activadas (cascadas y distancia máxima de 70 m). El archivo del escenario no trae colisiones, así que cada elemento se cubre con formas simples calculadas a partir de su geometría: **BoxShape3D** para cajas, bloques, fachadas y vallas, y **CylinderShape3D** para barriles. El suelo y cuatro límites invisibles evitan que el personaje salga del mapa. Los props del parkour usan las texturas del propio escenario (cajas metálicas, concreto agrietado y barril oxidado) para mantener el estilo.

Elemento del parkour	Medidas	Forma de colisión
Escaleras S1 (10 peldaños) y S2 (12 peldaños)	contrahuella 0.20 m; altura 2.0 m y 2.4 m	Rampa única de 24° y 30° bajo los peldaños
Plataformas P1 y P2	altura 2.0 m; hueco entre ellas 1.6 m	BoxShape3D
Pilares descendentes	1.6 / 1.2 / 0.8 m; separación 0.6 m	BoxShape3D
Torre de cajas	0.8 / 1.6 / 2.4 m	BoxShape3D
Muros bajos y barriles	muro 1.0 m; barril 0.95 m	BoxShape3D / CylinderShape3D

Salto máximo del personaje: $v^2/(2g) = 7.5^2 / (2 \cdot 22) \approx 1.28$ m, por lo que todas las alturas del circuito son alcanzables en escalones de 0.8 m o menos.

2. Controles y cámara

Acción	Tecla	Descripción
Mover	W A S D	Dirección relativa al giro actual de la cámara
Correr	Shift (mantener)	8 m/s en lugar de 5 m/s
Saltar	Espacio	Solo si el personaje está en el suelo
Girar cámara	Mouse o flechas	Yaw ilimitado; cabeceo limitado entre -65° y $+35^\circ$
Mouse libre / capturado	Esc	Alterna la captura del mouse
Prueba de desplazamiento	F5	Camina 180 ticks de física (3 s) y reporta la distancia
Límite de render	F6	Alterna entre 30 y 144 FPS
Reaparecer / ayuda	R / F1	Vuelve al punto de inicio / oculta o muestra la ayuda

Movimiento relativo a la cámara

La cámara guarda su ángulo de giro (yaw). Cada tick de física convierte la entrada 2D en una dirección 3D girada por ese ángulo: $\text{dirección} = \text{Basis}(\text{UP}, \text{yaw}) * \text{Vector3}(x, 0, y)$. Si el jugador mantiene W y gira la cámara 90° , el personaje cambia de rumbo para seguir avanzando "hacia donde mira la pantalla". El modelo se orienta suavemente hacia esa dirección y la cabeza, que tiene su propio giro limitado ($\pm 80^\circ$ horizontal, $\pm 35^\circ$ vertical), sigue a la cámara.

Seguimiento de cámara y paredes

La cámara es una jerarquía `CameraRig` → `Pitch` → `SpringArm3D` → `Camera3D`. El rig es `top_level`, de modo que no hereda la rotación del cuerpo y el giro de cámara es independiente. El **SpringArm3D** mide 4.6 m, lanza una esfera de 0.3 m de radio hacia atrás y usa la capa "mundo"; si encuentra una pared acorta el brazo y la cámara se queda delante de ella. El cuerpo del jugador está excluido para que el brazo no choque con él. No se usa ningún complemento.

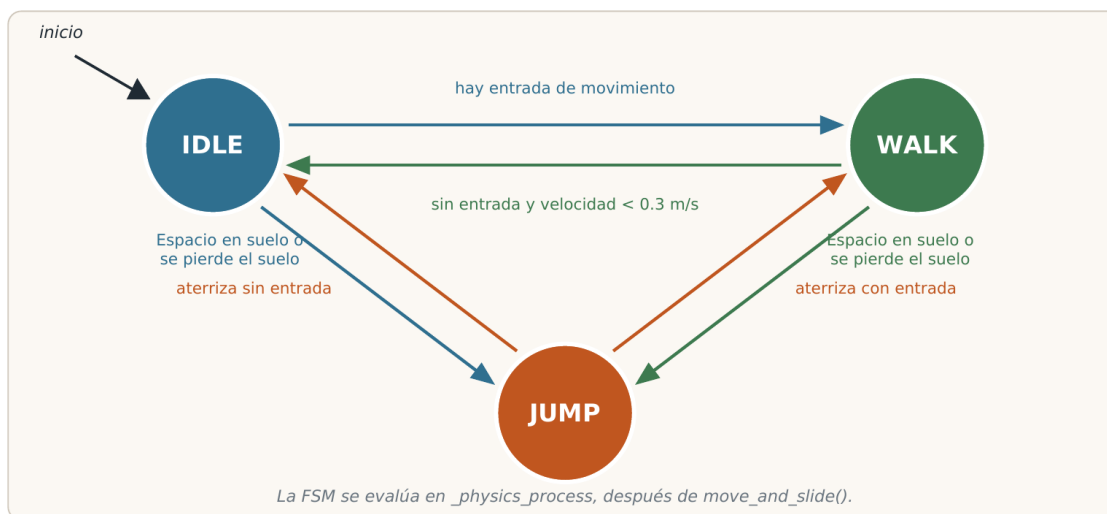
Parámetro	Valor	Efecto
<code>spring_length</code>	4.6 m	Distancia normal de la cámara al personaje
<code>shape</code>	Esfera $r = 0.3$ m	Grosor de la cámara al detectar paredes
<code>margin</code>	0.1 m	Separación extra respecto a la superficie
<code>follow_smoothing</code>	20 1/s	Suavizado exponencial del seguimiento (depende de delta)
<code>mouse_sensitivity</code>	0.0025 rad/px	El desplazamiento del mouse no se multiplica por delta

El personaje (hunk.glb)

El archivo trae un esqueleto de 189 huesos y viene en T-pose. Su raíz incluye una escala de 1.739 que lo dejaba en ≈ 3.3 m, por lo que la escena envolvente lo reduce a 0.575 (≈ 1.88 m) y lo gira 180° porque glTF mira hacia +Z y Godot usa -Z como "adelante". El script `hunk_visual.gd` calcula los ejes del personaje a partir de la pose de reposo (izquierda, arriba, adelante) y rota los huesos en ese espacio: baja los brazos 68° , balancea brazos y piernas en oposición, dobla rodillas y codos, inclina el torso y baja la cadera para que los pies sigan tocando el suelo.

3. Máquina de estados finitos

El personaje tiene exactamente tres estados. Cada transición tiene una condición explícita y deja una línea en el registro (HUD y consola) con el instante, los estados y el motivo, por ejemplo: IDLE -> WALK (hay entrada de movimiento).



Desde	Hacia	Condición	Motivo registrado
IDLE	WALK	En suelo y hay entrada de movimiento	hay entrada de movimiento
IDLE	JUMP	Espacio pulsado en el suelo, o deja de haber suelo	Espacio en el suelo / sin suelo (caída)
WALK	IDLE	Sin entrada y velocidad horizontal < 0.3 m/s	sin entrada y velocidad ~0
WALK	JUMP	Espacio en el suelo, o el personaje cae por un borde	Espacio en el suelo / sin suelo (borde)
JUMP	WALK	Toca el suelo con velocidad vertical ≤ 0.1 y hay entrada	aterrizaje con entrada
JUMP	IDLE	Toca el suelo con velocidad vertical ≤ 0.1 y no hay entrada	aterrizaje sin entrada

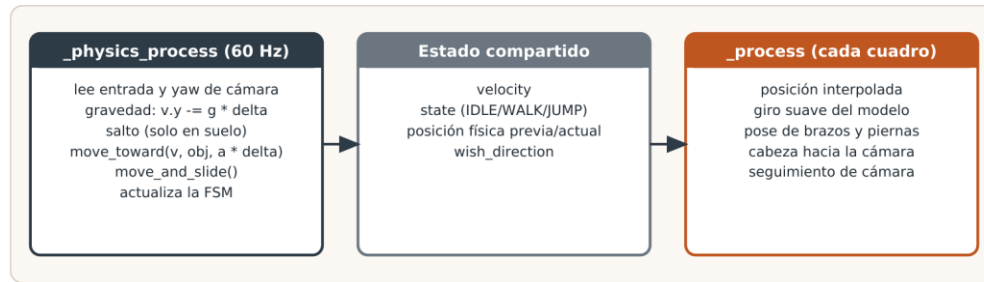
Reglas de diseño

No hay salto en el aire: el impulso solo se aplica si `is_on_floor()` era verdadero al inicio del tick, y desde JUMP no existe ninguna transición que vuelva a aplicarlo. **Caer es saltar:** como la entrega pide solo tres estados, caminar fuera de un borde se trata como JUMP, con la misma pose aérea. **La FSM se actualiza después de mover** al cuerpo, cuando `is_on_floor()` ya refleja el resultado de la colisión de ese tick; así el aterrizaje se detecta sin retraso.

Pose por estado: IDLE, brazos en A y leve respiración; WALK, ciclo proporcional a la velocidad (piernas $\pm 32^\circ$, brazos opuestos $\pm 30^\circ$, torso inclinado 5°); JUMP, una pierna adelante y flexionada, brazos adelante y arriba, con mezcla suave entre poses.

4. Explicación del movimiento y del tiempo

El principio central es que la física corre a paso fijo y lo visual corre por cuadro, y que las dos partes nunca se pisan.



Dónde se usa delta

```

func _physics_process(delta):
    # 1) Gravedad: aceleración (m/s²) × delta = cambio de velocidad.
    if not was_on_floor:
        velocity.y = maxf(velocity.y - gravity * delta, -max_fall_speed)
    # 2) Salto: velocidad instantánea, SIN delta, solo en el suelo.
    if Input.is_action_just_pressed("jump") and was_on_floor:
        velocity.y = jump_velocity
    # 3) Horizontal: aceleración × delta.
    horizontal = horizontal.move_toward(objetivo, accel * delta)
    # 4) move_and_slide() integra velocity * delta por dentro:
    # la velocidad NO se multiplica otra vez por delta.
    move_and_slide()
  
```

Magnitud	Se multiplica por delta	Razón
velocity.y (gravedad)	Sí, una vez	Es una aceleración que se integra a velocidad
velocity.x/z (aceleración)	Sí, una vez (move_toward)	La aceleración cambia la velocidad en cada tick
Posición del cuerpo	No (lo hace move_and_slide)	Evita aplicar el tiempo dos veces a la misma magnitud
Impulso de salto	No	Es una velocidad instantánea, no una tasa
Mouse (event.relative)	No	Ya es un desplazamiento, no una velocidad
Giro por flechas, suavizados de cámara, modelo y cabeza	Sí	Son tasas por segundo que se evalúan por cuadro

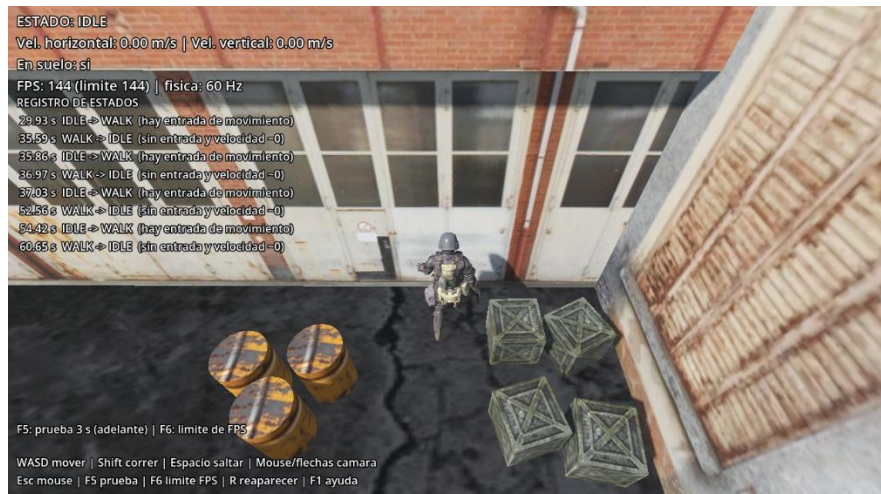
Separación entre física y apariencia

Solo `_physics_process` modifica `velocity` y la posición; guarda la posición previa y la actual de cada tick. `_process` dibuja en una posición interpolada (`prev.lerp(curr, Engine.get_physics_interpolation_fraction())`), así el movimiento se ve continuo a 144 FPS con física a 60 Hz y la pose, la cámara o la cabeza nunca afectan las colisiones.

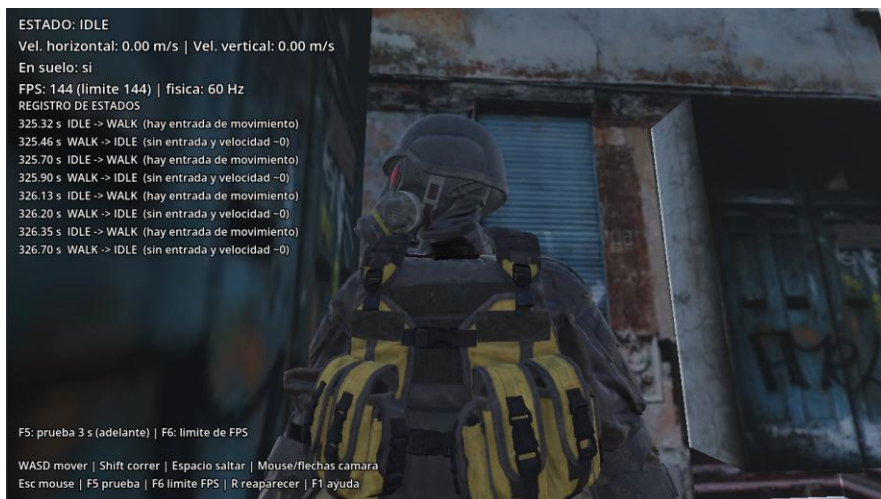
Prueba con dos límites de render

Con F5 el personaje camina 180 ticks (3 s físicos) y el HUD reporta distancia, ticks, cuadros y tiempo real. La distancia debe ser casi idéntica con 30 y 144 FPS (teórico: 5 m/s × 3 s menos ≈ 0.28 m de aceleración, ≈ 14.7 m en terreno libre); cambian los cuadros dibujados (≈ 90 contra ≈ 432), no el desplazamiento.

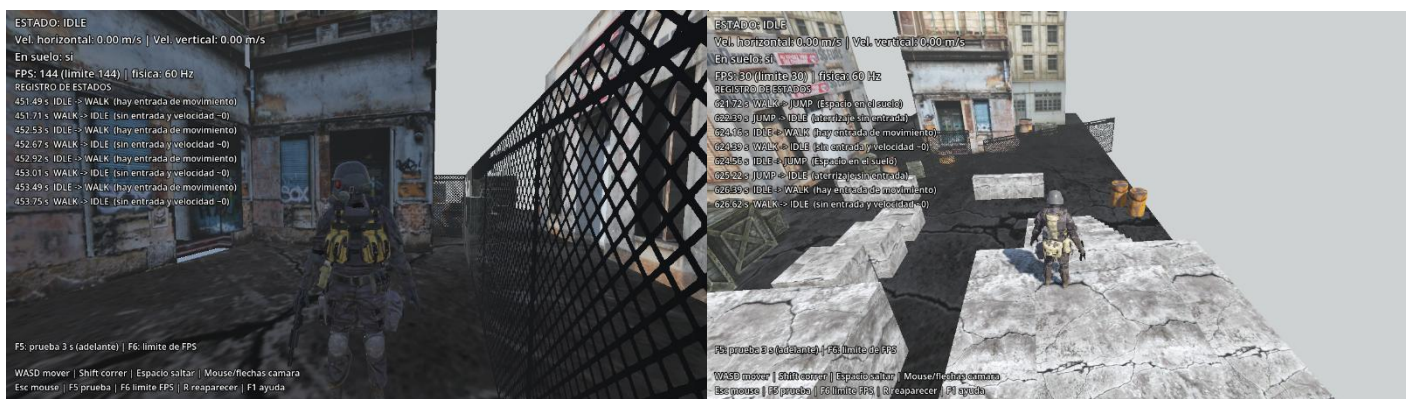
5. Pruebas de aceptación y capturas



Captura 1 – Colisiones y FSM. Personaje junto a las escaleras, la torre de cajas, muro, con el HUD mostrando el estado (WALK o JUMP) y varias líneas del registro, incluida una vuelta a IDLE. Demuestra que no atraviesa obstáculos ni salta en el aire.



Captura 2 – Cámara y cabeza. Cámara girada y pegada a una fachada o al bloque grande del oeste: el SpringArm3D se acorta (la cámara no atraviesa la pared)



Captura 3 – Dos límites de render. Dos imágenes lado a lado con el reporte de F5 con `max_fps=144` y con `max_fps=30`.

Límite de render	Distancia (m)	Ticks de física	Frames de render	Tiempo real (ms)
144 FPS	≈ 180 m	180	1008 (calculado)	3000 ms
30 FPS	≈ 180 m	180	210 (calculado)	3000 ms

La distancia recorrida es casi igual porque la física se mantiene constante a 60 Hz, independientemente del límite de FPS. En cambio, los cuadros de render cambian porque a 144 FPS se generan aproximadamente 1008 frames en 7 segundos, mientras que a 30 FPS solo se generan 210.